

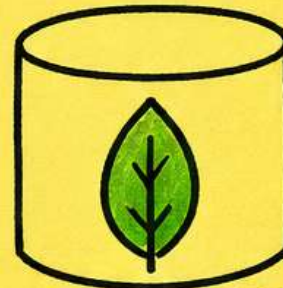
MongoDB



★ 1. What is MongoDB?

MongoDB is a **NoSQL document database**.

It stores data in flexible, JSON-like documents with dynamic schema.



mongoDB

2. NoSQL Database

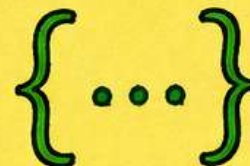
NoSQL = Not Only SQL.

It is designed for **high performance**, **scalability**, and **flexible schema**.

No Fixed Schema
Horizontal Scaling
High Availability

3. Key Features

- ✓ Document Oriented (**JSON-like**)
- ✓ Schema-less (**Dynamic Schema**)
- ✓ High Performance
- ✓ Highly Scalable
- ✓ Replication & High Availability
- ✓ Rich Query Language
- ✓ Indexing for faster queries



JSON-like Document

4. SQL vs MongoDB (Quick Comparison)

SQL (Relational DB)	vs	MongoDB (NoSQL)
Table	Structure	Collection
Row	Data Unit	Document
Column	Data Format	Key : Value (BSON)
Fixed Schema	Schema	Dynamic Schema
SQL	Query Language	MongoDB Query (MQL)
Vertical Scaling	Scaling	Horizontal Scaling



Think
JSON
Think
MongoDB!



MongoDB Architecture

★ 1. Database

A container for collections.
In MongoDB, a single instance can hold **multiple databases**.

★ 2. Collection

A group of MongoDB documents.
It is similar to a table in SQL.

★ 3. Document

A set of **key-value pairs**.
Documents are stored in **BSON** format.

★ 4. BSON

Binary JSON (BSON) is the storage format used by MongoDB.
It supports **more data types** than JSON and is faster to parse.

Hierarchy in MongoDB

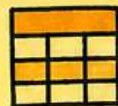
MongoDB Server



Database
(e.g., myDB)



Collection
(e.g., users)



Document
(e.g., { name: "Abhi", age: 24 })

Stored
as BSON

BSON
Example

```
{
  "_id" : ObjectId("64f2e1a9c1b3e2d1a6b7c8d9"),
  "name" : "Abhishek",
  "age" : 24,
  "skills" : ["Java", "MongoDB", "Node.js"],
  "isActive" : true,
  "createdAt" : ISODate("2024-05-04T10:20:30Z")
}
```



Key Takeaway :

MongoDB is schema-less and stores data as **documents** inside collections within databases on a MongoDB server.





MongoDB CRUD Operations

CRUD → Create, Read, Update, Delete

① CREATE (Insert)

Add new document(s) into a collection.



Syntax

```
db.collection.insertOne(  
  { field: value }  
)  
db.collection.insertMany([  
  {...}, {...}  
)
```

Example

```
db.users.insertOne({  
  name: "Abhi",  
  age: 24,  
  city: "Pune"  
})
```

② READ (Find)

Retrieve documents from a collection.



Syntax

```
db.collection.find()  
db.collection.find({ filter })  
db.collection.findOne({ filter })
```

Example

```
db.users.find({ age: { $gt: 18 } })  
// find all users age > 18  
db.users.findOne({ name: "Abhi" })  
// find one user
```

③ UPDATE

Modify existing documents.



Syntax

```
db.collection.updateOne(  
  { filter }, { $set: { field: value } }  
)  
db.collection.updateMany(  
  { filter }, { $set: { field: value } }  
)
```

Example

```
db.users.updateOne(  
  { name: "Abhi" },  
  { $set: { age: 25 } }  
)  
// update age of Abhi
```

④ DELETE

Remove documents from a collection.



Syntax

```
db.collection.deleteOne({ filter })  
db.collection.deleteMany({ filter })
```

Example

```
db.users.deleteOne({ name: "Abhi" })  
// delete one document  
db.users.deleteMany({ age: { $lt: 18 } })  
// delete all users with age < 18
```

Key Takeaway :

MongoDB provides simple and powerful methods to Create, Read, Update and Delete data.



MongoDB Query Operators

★ Query operators are used to **filter documents** in MongoDB.

1. Comparison Operators

Operator	Meaning	Example
\$eq	Equal to	{ age: { \$eq: 25 } }
\$ne	Not equal to	{ age: { \$ne: 25 } }
\$gt	Greater than	{ age: { \$gt: 25 } }
\$gte	Greater than or equal to	{ age: { \$gte: 25 } }
\$lt	Less than	{ age: { \$lt: 25 } }
\$lte	Less than or equal to	{ age: { \$lte: 25 } }
\$in	In an array	{ age: { \$in: [20, 25, 30] } }
\$nin	Not in an array	{ age: { \$nin: [20, 25, 30] } }

Example :

```
db.users.find({ age: { $gt: 18,
                      $lte: 30 } })
```



Find users with age greater than **18** and less than or equal to **30**.



2. Logical Operators

Operator	Meaning	Example
\$and	Joins query conditions with AND	{ \$and: [{ age: { \$gt: 18 } }, { city: "Pune" }] }
\$or	Joins query conditions with OR	{ \$or: [{ age: { \$lt: 18 } }, { city: "Delhi" }] }
\$not	Inverts the effect of expression	{ age: { \$not: { \$gt: 18 } } }
\$nor	Joins query conditions with NOR	{ \$nor: [{ age: { \$gt: 18 } }, { city: "Pune" }] }

AND vs OR

\$and → All conditions must be true

\$or → Any one condition can be true



3. Element Operators

Operator	Meaning	Example
\$exists	Checks if field exists	{ email: { \$exists: true } }
\$type	Matches BSON data type	{ age: { \$type: "number" } }
\$size	Matches array size	{ tags: { \$size: 3 } }
\$elemMatch	Matches array elements that satisfy all conditions	{ scores: { \$elemMatch: { \$gt: 80, \$lt: 100 } } }

Example :

```
db.users.find({ tags: { $size: 3 } })
```



Find users having tags array of size 3.



4. Evaluation / Other Operators

Operator	Meaning	Example
\$regex	Matches using regular expression	{ name: { \$regex: "^A" } } // names starting with A
\$text	Performs text search	{ \$text: { \$search: "MongoDB" } }
\$where	Matches documents that satisfy a JavaScript expression	{ \$where: "this.age > 18" }

Quick Tip

You can combine multiple operators in a single query to build **powerful filters**.



★ Mastering query operators = Writing **efficient** & **powerful** MongoDB queries!



MongoDB Projection & Sorting..

★ These operations help to **shape the results** returned by a query.

1 PROJECTION

Projection is used to include or **exclude** fields in the result.

Syntax:

```
db.collection.find(query, {  
  { field1: 1, field2: 0 } })
```

Example :

```
// Include only name and email  
db.users.find({}, { name: 1, email: 1,  
  ._id: 0 })  
  
// Exclude age field  
db.users.find({}, { age: 0 })
```



Select only what you need!

2 SORT()

sort() is used to sort the results in ascending or **descending** order.

Syntax:

```
db.collection.find(query)  
  .sort({ field: 1 | -1 })
```

Example :

```
// Ascending order (A to Z or 0 to 9)  
db.users.find().sort({ name: 1 })  
  
// Descending order (Z to A or 9 to 0)  
db.users.find().sort({ createdAt: -1 })
```

Ascending

A ↑ Z

Descending

Z ↓ A

3 LIMIT()

limit() specifies the **maximum number** of documents to return.

Syntax:

```
db.collection.find(query)  
  .limit(<number>)
```

Example :

```
// Get first 5 users  
db.users.find().sort({ name: 1 }).limit(5)
```



Limit the number of results!

4 SKIP()

skip() **skips** the specified number of documents.

Syntax:

```
db.collection.find(query)  
  .skip(<number>)
```

Example :

```
// Skip first 10 users  
db.users.find().sort({ name: 1 }).skip(10)
```

1 2 ... 10 11 12

↑
Skip first 10



How they work together?

You can **combine** them to get **exactly** what you want!

```
db.users.find({ age: { $gte: 18 } }, { name: 1, email: 1, _id: 0 })  
  .sort({ name: 1 })  
  .skip(10)  
  .limit(5)
```



Filter → Shape
→ Sort → Limit



Projection, sort, limit and skip = **Power tools** for efficient and clean queries!



MongoDB Indexing..



★ Indexes improve the **speed of queries**.

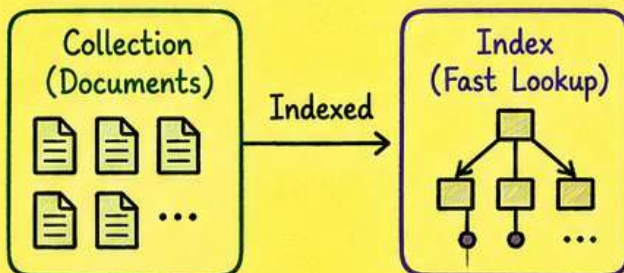
Without index, MongoDB scans every document.
With index, it finds data quickly.

Think of index
like the index of
a book!



1 What is Index?

An index is a special data structure that stores a small portion of the collection's data in an **easy-to-traverse** form.



2 Why Index?

- ✓ Speeds up **read** operations.
- ✓ Improves sorting and filtering.
- ✓ Supports **uniqueness** constraints.
- ✓ Optimizes aggregation pipelines.
- ✓ Reduces the load on the server.

Without Index
Scan all documents



With Index
Quick lookup



3 Types of Indexes

1. Single Field Index

Index on a single field.

Example:

```
db.users.createIndex({ name: 1 })
```



2. Compound Index

Index on multiple fields.

Example:

```
db.users.createIndex({ name: 1, age: -1 })
```

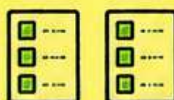


3. Multikey Index

Index on array fields.

Example:

```
db.users.createIndex({ tags: 1 })
```



4. Unique Index

Ensures field values are unique.

Example:

```
db.users.createIndex({ email: 1 }, { unique: true })
```



5. Text Index

Supports text search.

Example:

```
db.articles.createIndex({ content: "text" })
```



4 Index Commands

- Create Index → `db.collection.createIndex({ field: 1 })`
// 1 = Ascending, -1 = Descending
- View Indexes → `db.collection.getIndexes()`
- Drop Index → `db.collection.dropIndex("indexName")`
- Drop All Indexes → `db.collection.dropIndexes()`

5 Pros & Cons

Pros

- ✓ Faster query performance
- ✓ Improved sorting
- ✓ Efficient aggregation
- ✓ Better scalability

Cons

- ✗ Takes extra storage
- ✗ Slower write operations
- ✗ More indexes = more overhead



Key Takeaway :

Use indexes wisely.
Index the fields you query and **sort** the most,
but avoid **too many indexes**.



MongoDB Aggregation Framework

★ The Aggregation Framework is a **powerful tool** used to process data and return computed results.

Think of it as a data processing **pipeline!**

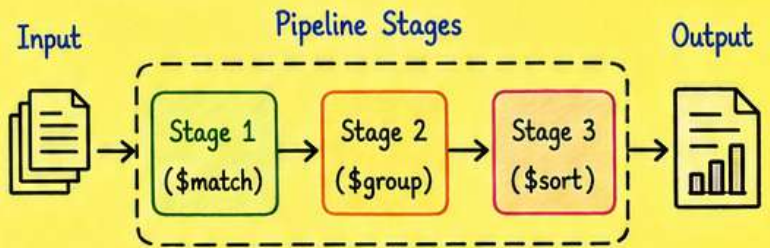
① What is Aggregation?

Aggregation allows you to perform operations on data and return a single result. It works with a **pipeline of stages**.

Syntax :

```
db.collection.aggregate([
  { stage1: { ... } },
  { stage2: { ... } },
  ...
])
```

② Aggregation Pipeline



Data flows through each stage and is transformed. The final **output** is returned.

③ Common Aggregation Stages

Stage	Purpose	Example
\$match	Filters documents (like WHERE in SQL)	<code>{ \$match: { age: { \$gt: 18 } } }</code>
\$project	Reshapes documents (like SELECT in SQL)	<code>{ \$project: { name: 1, age: 1, _id: 0 } }</code>
\$group	Groups documents (like GROUP BY in SQL)	<code>{ \$group: { _id: "\$city", total: { \$sum: "\$salary" } } }</code>
\$sort	Sorts documents (like ORDER BY in SQL)	<code>{ \$sort: { age: -1 } }</code>
\$limit	Limits the number of documents	<code>{ \$limit: 5 }</code>
\$skip	Skips the specified number of documents	<code>{ \$skip: 10 }</code>
\$unwind	Deconstructs an array field	<code>{ \$unwind: "\$tags" }</code>
\$lookup	Performs left outer join with another collection	<code>{ \$lookup: { from: "orders", localField: "_id", foreignField: "userId", as: "orders" } }</code>



Each stage takes the **output** from the previous stage and passes it to the next stage.

④ How it Works? (Example Flow)

Input Data (Collection)



Output (Result)



Example Pipeline :

```
db.sales.aggregate([
  { $match: { status: "A" } },           // 1. Filter
  { $group: { _id: "$product", total: { $sum: "$amount" } } },
  { $sort: { total: -1 } },              // 3. Sort
  { $limit: 5 }                          // 4. Limit
])
```

What this does?

- Filters only documents with status = "A"
- Groups by product and calculates **total amount**
- Sorts by total in descending order
- Returns top 5 results



Sample Output

_id (product)	total
Laptop	25000
Phone	18000
Tablet	12000
Watch	8000
Earbuds	5000
...	...



Key Takeaway :

Aggregation Framework helps you **analyze**, transform and **summarize** data in powerful ways.



Master aggregation, master your **data!**



MongoDB Data Modeling..



★ Good data modeling in MongoDB means storing data in the **right way** to get better performance and **scalability**.

Two Main Approaches : 1. **Embedding** 2. **Referencing**

① Embedding

Store related data inside the **same document**.

Use When :

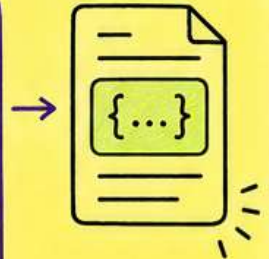
- Data is closely related
- One-to-Few relationship
- Data is read together more often
- Size is not too large

Example

A blog with comments.

```
{
  _id: ObjectId("1"),
  title: "MongoDB Tips",
  author: "Abhi",
  comments: [
    { user: "Rohit", text: "Great post!", date: ISODate("2024-05-01") },
    { user: "Neha", text: "Very helpful!", date: ISODate("2024-05-02") }
  ]
}
```

Single Document



② Referencing

Store related data in **separate** documents and link them using references.

Use When :

- Data grows large
- Many-to-Many relationship
- Data is updated independently
- Need to reuse data in multiple places

Example

A blog with separate comments collection.

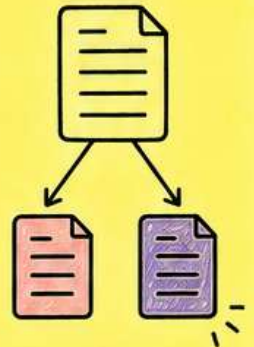
Collection: blogs

```
{
  _id: ObjectId("1"),
  title: "MongoDB Tips",
  author: "Abhi",
  commentIds: [
    ObjectId("c1"),
    ObjectId("c2")
  ]
}
```

Collection: comments

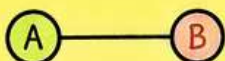
```
{
  _id: ObjectId("c1"),
  blogId: ObjectId("1"),
  user: "Rohit",
  text: "Great post!",
  date: ISODate("2024-05-01")
}
-----
{
  _id: ObjectId("c2"),
  blogId: ObjectId("1"),
  user: "Neha",
  text: "Very helpful!",
  date: ISODate("2024-05-02")
}
```

Multiple Documents



③ Relationship Patterns

One-to-One

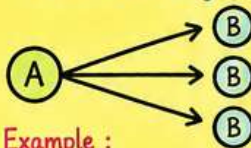


Example :

User ↔ UserProfile

Store either embedded or referenced.

One-to-Many

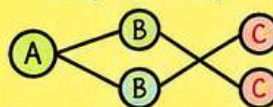


Example :

Blog → Comments

Embed if few,
Reference if many.

Many-to-Many

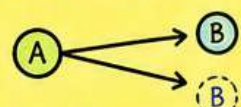


Example :

Students ↔ Courses

Use referencing with
junction collection.

One-to-Few (Subset)



Example :

Order → Items (few)

Embed when count
is small.



Key Takeaway : Model your data based on how you read and write it.

Good modeling = Better Performance + Scalability + Happy Developers 😊



MongoDB Replication..



★ Replication in MongoDB provides **high availability** and data redundancy.

It is achieved using Replica Sets.

If one server fails, another takes over **automatically!** 😊

Benefits

- ✓ High Availability
- ✓ Automatic Failover
- ✓ Data Redundancy
- ✓ Scalability for Read Operations
- ✓ Zero Downtime for Maintenance

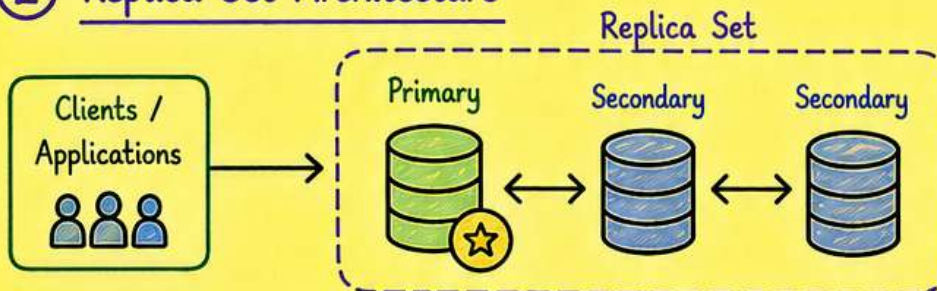


① What is Replica Set?

A Replica Set is a group of **mongod** instances that maintain the same data.

It has one **primary** and **multiple secondaries**.

② Replica Set Architecture

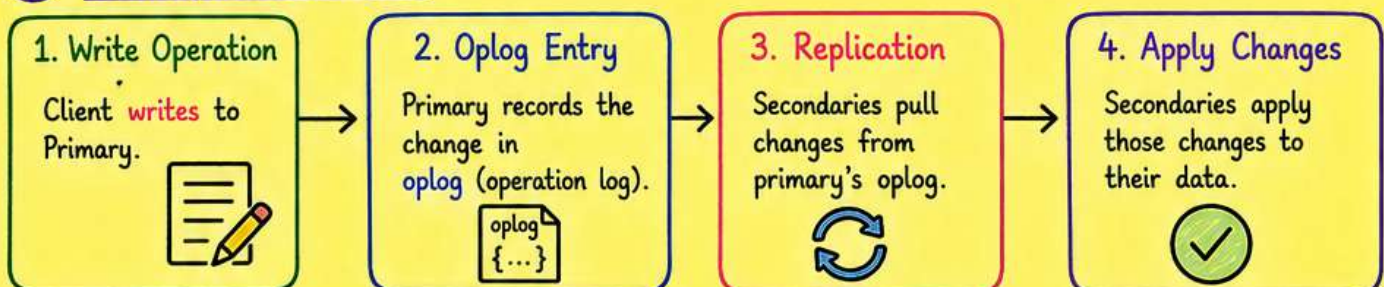


- **Primary** → Handles all **write** operations.
- **Secondaries** → Replicate data from primary.
- **Clients** → Read from primary by default, or from secondaries (**read preference**).

Read Preference

- **primary** → Read from primary
- **secondary** → Read from any secondary
- **nearest** → Read from nearest node
- **secondaryPreferred** → Read from secondary if available

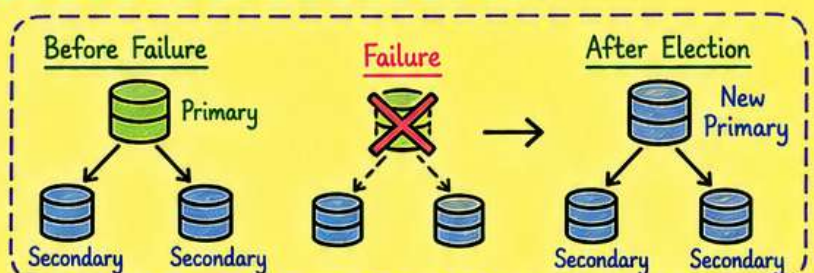
③ How Replication Works?



Note: Secondaries are **eventually consistent** (very small delay).

④ Automatic Failover

- If primary fails, the replica set holds an **election**.
- One of the secondaries becomes the new primary.
- The process is **automatic** and transparent to clients.
- Ensures high availability.



Key Takeaway: Replica Sets = High Availability + Data Safety + Peace of Mind! 😊

MongoDB Sharding..

- ★ Sharding is the process of **distributing data** across **multiple machines (shards)** to handle **large datasets** and **high throughput**.

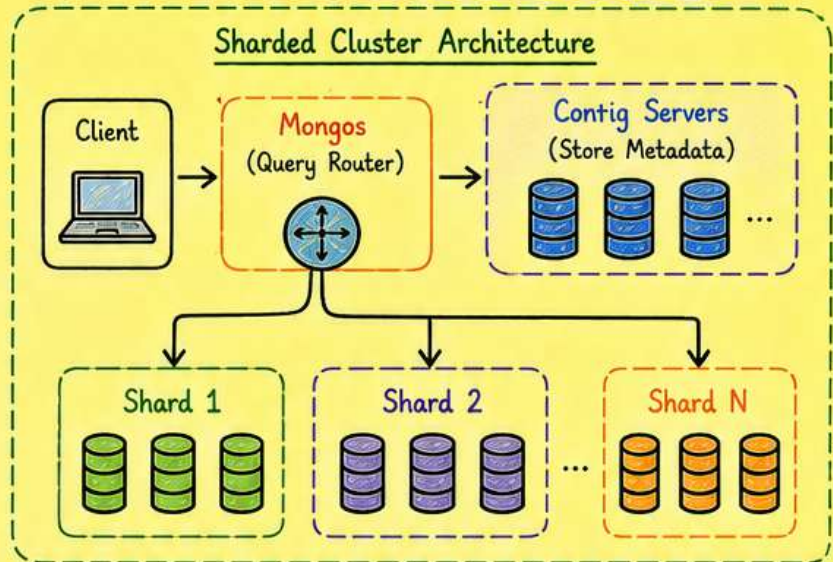
Think of it like splitting a big collection into smaller pieces and storing them on **different servers!** 😊

① What is Sharding?

Sharding allows MongoDB to **scale horizontally** by partitioning data across multiple shards. Each shard is a **replica set** to ensure **availability** and **fault tolerance**.

Key Points

- Data is split based on a **shard key**.
- Each shard can be a **replica set**.
- **Mongos** routes queries to the right shard(s).
- **Config servers** store cluster metadata.



② Shard Key

A shard key is a field (or combination of fields) used to **distribute** data across shards.

Example

```
// Shard collection 'orders' on 'userId'
sh.shardCollection("ecommerce.orders", { userId: 1 })

// Hashed shard key (good for even distribution)
sh.shardCollection("ecommerce.orders", { userId: "hashed" })
```

Good Shard Key

- ✓ Has high cardinality
- ✓ Evenly distributed
- ✓ Frequently queried
- ✓ Immutable (rarely changes)

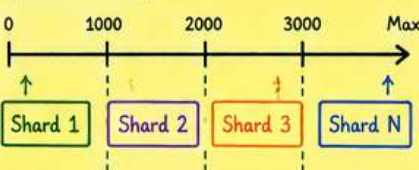
③ Sharding Strategies

1. Range-based Sharding

- Data is split by ranges of shard key values.

Example:

```
{ "userId": 1 }
```



2. Hashed Sharding

- Applies a hash function to shard key and distributes evenly.

Example:

```
{ "userId": "hashed" }
```

userId	Hash Value	Shard
101	a3f2..	Shard 2
102	9b7d..	Shard 1
103	1c8e..	Shard 3
104	f4a1..	Shard N
105	0d9c..	Shard 2
...	...	...

3. Zone Sharding

- Shards are assigned to zones (e.g., regions).

Example:

Zone 1 (US) → Shard 1
 Zone 2 (EU) → Shard 2
 Zone 3 (ASIA) → Shard 3



④ Pros & Cons

Pros



- ✓ Horizontal Scalability
- ✓ Handles Large Datasets
- ✓ High Throughput
- ✓ Fault Tolerance

Cons



- ✗ Complex to Setup
- ✗ More Operational Overhead
- ✗ Transactions across shards are limited



Key Takeaway

Sharding helps MongoDB **scale horizontally** by splitting data across multiple shards. Choose the **right shard key** and strategy for your use case! 😊



MongoDB Sharding Deep Dive..

★ Sharding is one of MongoDB's most **powerful features** for scaling large datasets and handling high load.

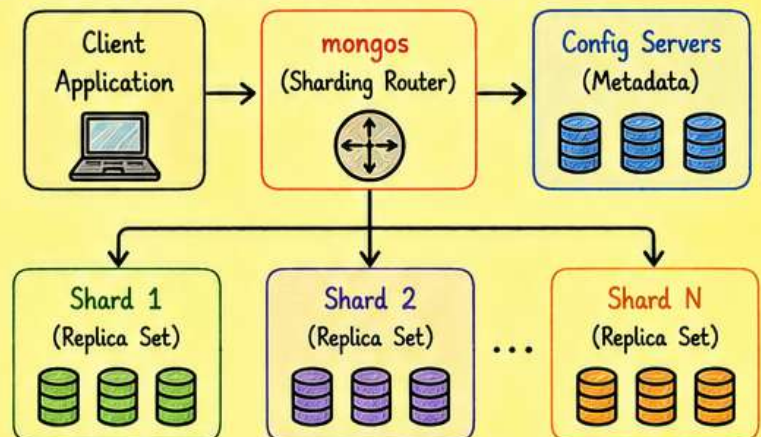
Sharding =
Scale Out with
Smart Distribution!



① Sharding Components

	Sharding Router (mongos)	Routes client requests to the right shard.
	Config Servers	Store metadata about the cluster and chunks.
	Shards	Store the actual data. Each shard is a replica set.
	Client	Sends requests to mongos.

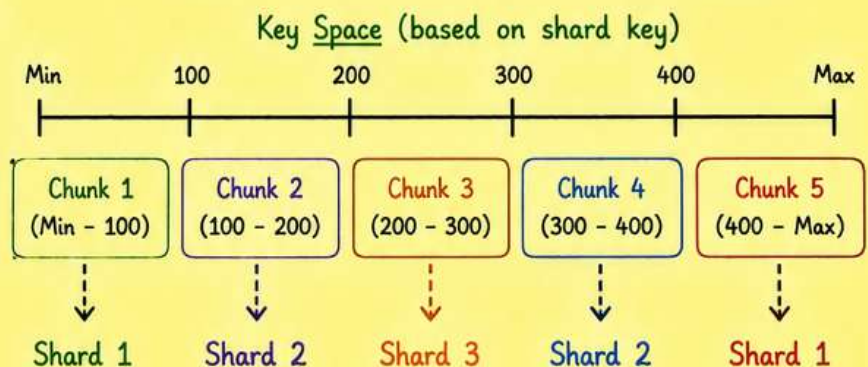
② Request Flow in a Sharded Cluster



mongos uses metadata from Config Servers to **route queries** to the correct shard(s).

③ Chunk & Chunk Distribution

A shard stores a range of data called a **chunk**. The entire **key space** is divided into chunks and distributed across shards.



Important

- Chunks can be **moved** between shards.
- Helps in **load balancing** automatically.

④ Example: Sharding on "userId"

A. Enable Sharding

```
// Enable sharding on a database
sh.enableSharding("ecommerce")

// Shard a collection on userId
sh.shardCollection("ecommerce.orders", { userId: 1 })
```

C. View Shard Distribution (Chunks)

```
sh.status() // Shows cluster status
db.orders.getShardDistribution()
db.orders.getSplitKeysForChunks()
```



B. Insert Sample Data

```
db.orders.insertMany([
  { userId: 1, item: "Laptop", amount: 70000 },
  { userId: 2, item: "Phone", amount: 20000 },
  { userId: 3, item: "Headphones", amount: 2000 },
  { userId: 4, item: "Keyboard", amount: 1500 },
  ...
])
```

D. Add More Shards (Scale Out)

```
// Add a new shard
sh.addShard("shard4/host1:27017,host2:27017,host3:27017")

// Balancer will move chunks automatically
sh.startBalancer()
```



Key Takeaway:

Sharding allows MongoDB to handle **massive scale** by distributing data across shards. Choose the **right shard key** and let MongoDB balance the load for you!





MongoDB Sharding Deep Dive..

Sharding =
Speed + Scale
+ Smart Work!



- ★ Sharding helps MongoDB handle **huge data**, **high traffic** and keep your application fast and available.

5 Choosing the Right Shard Key

Good Shard Key

- ✓ High cardinality (many unique values)
- ✓ Evenly distributed
- ✓ Frequently used in queries
- ✓ Immutable (doesn't change)
- ✓ Not too large

Bad Shard Key

- ✗ Low cardinality (few unique values)
- ✗ Skewed distribution
- ✗ Random / Unpredictable
- ✗ Mutable (changes often)
- ✗ Too large (big documents)

Example

```
// Good
sh.shardCollection("ecommerce.orders", { userId: 1 })
// userId has many unique values

// Bad
sh.shardCollection("ecommerce.orders", { status: 1 })
// status has few values (placed, shipped, etc.)

// Bad
sh.shardCollection("ecommerce.orders", { createdAt: "hashed" })
// time based key can be uneven
```



6 Balancing & Chunk Migration

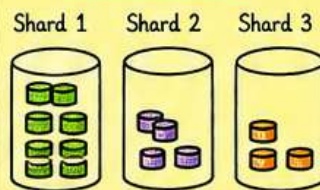
MongoDB **automatically** balances data across shards by moving chunks from **busy** shards to **less busy** shards.

Note

- Balancer runs in the background.
- You can start / stop balancer.
- Helps keep load evenly distributed.

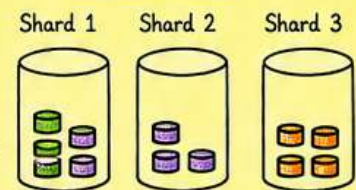


Before Balancing



Uneven Data
(Shard 1 is overloaded)

After Balancing



Even Data Distribution
(Balanced Load)



Useful Commands

- `sh.startBalancer()` → Start auto balancing
- `sh.stopBalancer()` → Stop auto balancing
- `sh.getBalancerState()` → Check balancer status



7 Write Operations in Sharded Cluster

- Writes go to the **owning shard** of the data.
- If the shard key is in the query filter, mongos routes the write to the correct shard.
- If not, mongos may need to query multiple shards.

Example

```
// Good (shard key in filter)
db.orders.updateOne({ userId: 101 }, { $set: { status: "shipped" } })

// Bad (no shard key in filter → may scatter to all shards)
db.orders.updateMany({ status: "placed" }, { $set: { status: "shipped" } })
```

8 Read Operations in Sharded Cluster

- Reads are routed by **mongos** based on the query.
- If the shard key is in the query, only **relevant shard(s)** are queried.
- If not, mongos queries all shards (scatter-gather).

Example

```
// Good (targets one shard)
db.orders.find({ userId: 101 })

// Bad (scatter-gather to all shards)
db.orders.find({ status: "placed" })
```



9 Monitoring & Best Practices

Best Practices

- ✓ Choose the right shard key
- ✓ Monitor chunk distribution
- ✓ Keep shards balanced
- ✓ Avoid jumbo chunks
- ✓ Scale config servers for large clusters
- ✓ Test your queries!



Useful Commands

- `sh.status()` → Cluster overview
- `db.orders.getShardDistribution()` → Collection distribution
- `db.orders.getSplitKeysForChunks()` → Chunk ranges
- `sh.getCollectionBalancingStatus("db.orders")` → Balancing status
- `sh.getConfigServers()` → Config server info

Things to Monitor

- ★ Chunk distribution
- ★ Shard storage & CPU
- ★ WiredTiger cache
- ★ Connections
- ★ Slow queries



Key Takeaway : Sharding is MongoDB's superpower for scaling. Use the right shard key, **monitor** your cluster and let MongoDB handle the scale!





MongoDB Sharding Deep Dive..

- ★ Sharding makes MongoDB **horizontally scalable**, **fault tolerant** and **highly available**.

Good Design +
Right Strategy =
Blazing Fast App!



10) Shard Key Best Practices

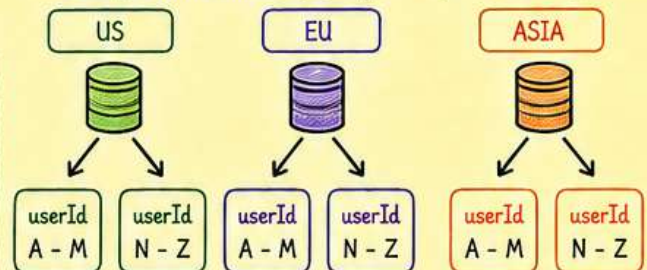
- ✓ Choose fields with high cardinality.
- ✓ **Avoid** monotonically increasing fields (e.g. timestamp, auto-increment id).
- ✓ Choose fields that are part of your query filter.
- ✓ Keep shard key immutable (don't update it).
- ✓ Use compound shard key if single field is not enough.

Good vs Bad Shard Key

Good 🍀	Bad 🚫
• userId • productId • region • category + userId	• createdAt (timestamp) • _id (ObjectId) • status (few unique) • sequential numbers



Example Distribution ({ region: 1, userId: 1 })



11) Compound Shard Key

Use **multiple fields** as shard key to get better distribution.

Example

```
sh.shardCollection("orders",
  { region: 1, userId: 1 })
```

How it Works?

- MongoDB first uses 'region' to split data.
- Inside each region, it uses 'userId'.



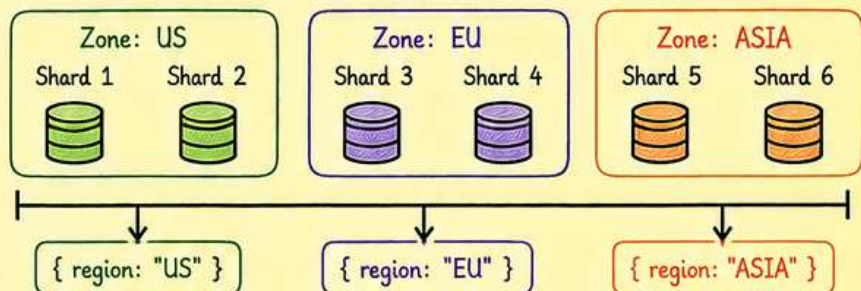
Compound shard key → Better distribution when one field has limited cardinality.

12) Zones in Sharding

Zones allow you to **pin shards** to specific regions or data centers.

Example

```
// Add zones
sh.addShardToZone("shard1", "US")
sh.addShardToZone("shard2", "US")
sh.addShardToZone("shard3", "EU")
...
// Add ranges to zones
sh.updateZoneKeyRange("orders",
  { region: "US" }, { region: "US" })
```



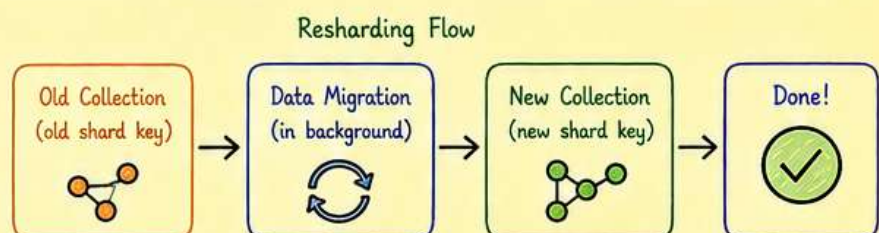
Zones help in **data locality**, **compliance** and **disaster recovery**.

13) Resharding (MongoDB 5.0+)

Resharding lets you change the shard key of a collection **without downtime**.

Steps

- 1 Create a new collection with new shard key
- 2 MongoDB migrates data in background
- 3 Switches the collection
- 4 Old collection is dropped automatically



No downtime for your application! 😊

14) When NOT to Use Sharding?



Small dataset
(fits in a single server)



Low write/read
traffic

Need for strong
multi-document
transactions



Complex aggregations
across all data
(scatter-gather cost)



High operational
overhead not
affordable



Key Takeaway: Use sharding only when you really need to scale.

Design it right, **monitor** it well and MongoDB will **scale like a champ!**





MongoDB Sharding Deep Dive..

★ Sharding is not just about splitting data, it's about designing for **Performance**, **Scalability** and **Reliability**.

Right Design
+ Right Monitoring
= **Rockstar System!**



15 Sharding Use Cases

- ✓ Large datasets that exceed single server capacity.
- ✓ High write/read throughput requirements.
- ✓ Geographically distributed users.
- ✓ Multi-tenant (separate data per tenant).
- ✓ Analytics + Operational workload separation.

Example Scenarios

- E-commerce Orders (millions of orders/day)
- Social Media Feeds (billions of events)
- IoT Data (high velocity sensor data)
- SaaS Applications (tenant based isolation)

16 Sharding Limitations

- ✗ Joins across shards are not supported.
- ✗ Transactions across shards are expensive.
- ✗ Hot shards can still become bottlenecks.
- ✗ Complexity in setup, monitoring and operations.
- ✗ Not suitable for small datasets.

Tip:

Sharding is powerful, but comes with operational complexity. **Plan wisely!**



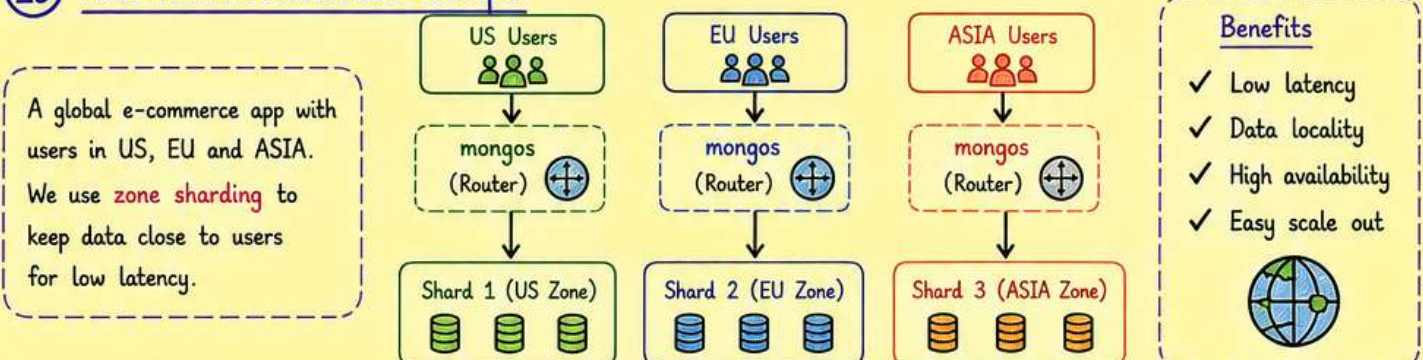
17 Sharding vs Replication

Feature	Sharding	Replication
Purpose	Scale Horizontally	High Availability
Data	Split across servers	Same data copied
Reads	Distributed	From secondaries
Writes	Distributed	Primary only
Failure Handling	Shard level	Replica level
Complexity	High	Low / Medium

18 Sharding Design Checklist



19 Real World Architecture Example



Key Takeaway:

Sharding is a journey from Monolith to Distributed. Design it right, **operate** it smart and **scale** with confidence!





MongoDB Sharding Deep Dive..

- ★ Great design + smart operations + continuous monitoring
= A **Scalable, Fast & Reliable** MongoDB System!

Plan Today,
Scale Tomorrow,
Succeed Always!



20 Tips from Real World

- ★ Start small, shard when you really need to.
- ★ Always test your shard key with real data.
- ★ Monitor chunk distribution regularly.
- ★ Keep chunks balanced for best performance.
- ★ Use automation (balancer) but validate results.
- ★ Watch your slow queries and optimize indexes.
- ★ Plan for growth, not just for today.
- ★ Document your strategy and share with team.

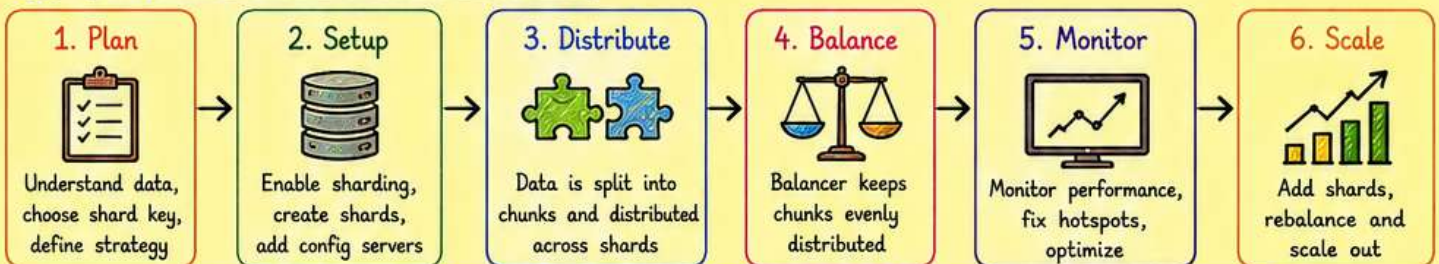
Remember

- ✓ Shard Key → The Heart
- ✓ Chunks → The Building Blocks
- ✓ Balancer → The Helper
- ✓ Monitoring → The Guardian
- ✓ Good Design → The Success

Design Right. Scale Right. Win Big! 😊



21 Complete Flow: From Start to Scale



Repeat 4 → 5 → 6 continuously to handle growth and keep system healthy!



22 MongoDB Sharding Cheat Sheet

Task	Command / Concept	Description
⚙️ Enable Sharding	<code>sh.enableSharding("<db>")</code>	Enables sharding on a database
📊 Shard Collection	<code>sh.shardCollection("<db>.<coll>", { key: 1 })</code>	Shards a collection on the given key
🔑 Hashed Shard Key	<code>{ key: "hashed" }</code>	Distributes data evenly using hash
📍 Add Shard	<code>sh.addShard("shardHost:port")</code>	Adds a new shard to the cluster
📈 View Shards	<code>sh.status()</code>	Shows cluster status and shards
📊 Chunk Distribution	<code>db.<coll>.getShardDistribution()</code>	Shows chunk distribution
↕️ Chunk Ranges	<code>db.<coll>.getSplitKeysForChunks()</code>	Shows chunk ranges
⚖️ Start Balancer	<code>sh.startBalancer()</code>	Starts the balancer
⏸️ Stop Balancer	<code>sh.stopBalancer()</code>	Stops the balancer
⚙️ Balancer Status	<code>sh.getBalancerState()</code>	Checks if balancer is running
📋 Config Servers	<code>sh.getConfigServers()</code>	Shows config server info
📁 Add Zone	<code>sh.addShardToZone("shard", "zone")</code>	Assigns shard to a zone
📁 Update Zone Range	<code>sh.updateZoneKeyRange("db.coll", { min: ... }, { max: ... }, "zone")</code>	Defines range for a zone
❌ Remove Shard	<code>sh.removeShard("shardHost:port")</code>	Safely removes a shard
📄 Explain Query	<code>db.<coll>.explain("executionStats").find(...)</code>	Analyzes query performance



Final Takeaway :

Sharding is MongoDB's **superpower** for scale.

Understand your data, choose the right shard key,

monitor smartly, and your app will be **future-ready!** 🚀

